

ISA 语义与数据通路

从软件承诺到硬件实现需求的接口契约

408 · 计算机组成原理 Bridge CO-B01

母接口

CO-02 输出一条指令对软件承诺的状态变化；CO-03 需要的却不是助记符，而是一组可实现、可检查的值与状态需求。两侧通过一个规范化语义包交接：

Instruction Semantic → Architectural Delta Packet → Value Dependency Contract → Datapath / Control Inp

箭头表示接口翻译，不表示时钟周期。Bridge 只拥有“左侧输出怎样变成右侧合法输入”，不拥有指令定义或具体通路。

Owners 与 Stop Boundary

- **Left Owner:** CO-02 Own ISA contract、编码、EA、访存宽度、Next PC 与异常语义。
- **Right Owner:** CO-03 Own 值依赖图、路径、资源调度、控制字、关键路径和提交。
- **Bridge Owns:** 规范化状态差、接口字段完整性、语义到实现需求的翻译顺序与双向校验。
- **Stops:** 不重讲 C/ABI，不选择具体 MUX/总线，不讨论多指令 hazard；后两者分别留给 CO-03/04。

目录

1 为什么需要独立接口	1
2 左侧输出：架构状态差包	1
3 翻译层：从状态差到值依赖契约	2
4 最小例一：ADD	2
5 最小例二：LOAD	3
6 接口不变量与双向验证	3
7 调用时机与停止条件	3
8 Anti-Bridge：必须停止的类比	4
9 一页压缩	4

1 为什么需要独立接口

若 CO-02 只给助记符，CO-03 会被迫猜“到底读什么、写什么、异常时哪些效果禁止提交”；若 CO-03 直接从教材图反推语义，又会把一种实现误写成 ISA。接口必须把“软件承诺”转换成与具体微架构无关、却足以生成实现的需求。

这条接口会在三类问题中重复调用：为已有 CPU 增加指令、判断现有通路是否足够、把单指令语义交给流水线/Integration。它因此不是单向 Use 的一句摘要。

2 左侧输出：架构状态差包

对指令 I 和初始架构状态 A ，CO-02 应输出

$$\Delta_I = \langle Read, Derived, Write, NextPC, Memory, Exception \rangle.$$

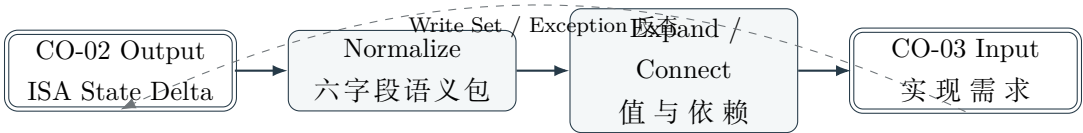
字段	必须回答	接口失败的症状
Read Set	哪些架构值是语义输入	通路少读源或误读目的寄存器
Derived Values	EA、算术结果、条件、扩展值等	不知道应放哪种功能部件
Write Set	哪些架构位置允许改变	多余写使能造成错误副作用
Next PC	顺序、分支、跳转和返回目标	PC 来源或选择条件不完整
Memory Contract	读/写、宽度、地址、扩展、对齐	LOAD/STORE 方向或宽度错误
Exception Contract	检测条件、允许提交的效果、重试语义	faulting 指令留下部分副作用

这个包不包含“第几拍做”“使用几个 ALU”“是否有 MAR/MDR”。这些属于右侧实现选择。

3 翻译层：从状态差到值依赖契约

Bridge 依次执行五步：

- 1. **Normalize**：把助记符、字段和寻址方式还原成状态转移，不保留模糊的“执行一下”。
- 2. **Expand**：把复合表达式拆为具名中间值，例如 $EA = base + sext(imm)$ 、 $data = M[EA]$ 。
- 3. **Connect**：为每个值写 producer、consumer 和先后依赖。
- 4. **Protect**：把 Write Set 与 Exception Contract 转成禁止副作用约束。
- 5. **Hand off**：只把值、依赖、允许写入和 memory/exception contract 交给 CO-03。



4 最小例一：ADD

CO-02 给出

$$\begin{aligned} Read &= \{R[rs_1], R[rs_2], PC\}, \quad Derived = \{sum, PC_{seq}\}, \\ Write &= \{R[rd], PC\}, \quad R'[rd] = R[rs_1] + R[rs_2]. \end{aligned}$$

Bridge 将它翻译为两个生产链：寄存器读 → 加法 → 目标写回，以及 PC → 顺序地址 → PC 写回。CO-03 再决定这些链由一个长周期、多个周期还是其他通路完成。

接口检查

若实现打开 MemWrite，Write Set 立即否决；若实现只读取一个源寄存器，Read Set 立即否决。校验不依赖某张标准图。

5 最小例二：LOAD

对抽象指令 `LOAD rd, imm(rs1)`，左侧语义包为

$$\begin{aligned} Read &= \{R[rs1], imm, PC\}, \\ EA &= R[rs1] + sext(imm), \\ data &= M[EA, w], \\ Write &= \{R[rd], PC\}. \end{aligned}$$

Bridge 生成依赖

$$R[rs1], imm \rightarrow EA \rightarrow memory\ request \rightarrow data \rightarrow extension \rightarrow R[rd].$$

同时附带：内存必须是读请求、宽度为 w 、目标写回只能在数据可用且无异常时发生。存储系统何时返回由接口信号表示，不在 Bridge 假定固定一拍。

为什么这不是 CO-03 的重复正文

Bridge 只声明 CO-03 必须收到哪些值、依赖和禁止副作用；是否复用 ALU、是否需要临时寄存器、怎样排拍和控制信号为何，是 CO-03 的完整责任。

6 接口不变量与双向验证

- **Completeness**: 每个 ISA 输入都出现在 Read Set 或明确的隐含状态中；每个允许结果都出现在 Write Set。
- **No invention**: 实现需求不能新增 ISA 未规定的架构副作用。
- **Dependency preservation**: consumer 只能使用已经由 producer 产生的值。
- **Exception safety**: 异常路径不得提交被契约禁止的部分效果。
- **Implementation freedom**: 接口不限制具体部件数量、拍数或内部寄存器命名。

完成设计后进行双向检查：从 ISA 向下确认每个语义效果有实现路径；从通路向上确认每个写使能都能在 Write Set 或 Next PC 中找到授权。

7 调用时机与停止条件

题面信号	调用动作	何时退出 Bridge
“增加一条新指令”	建六字段语义包并展开值依赖	进入选部件/加 MUX 时交给 CO-03
“判断通路能否支持”	对照 Read/Derived/Write/NextPC	开始排拍和算关键路径时退出
“写微操作序列”	先验证序列覆盖语义包	具体总线冲突与控制字交给 CO-03
“流水线中该指令如何走”	提供 producer/consumer 与提交条件	多指令 need/ready/hazard 交给 CO-04

8 Anti-Bridge：必须停止的类比

- **ISA 字段 ≠ 硬件连线：** 字段说明语义来源，不保证直接连到某部件。
- **微操作 ≠ 指令语义：** 微操作是实现步骤，不能反向成为所有机器的 ISA 定义。
- **Ready ≠ Commit：** 值已经产生不表示架构写入已发生。
- **C 语句 ≠ 唯一指令序列：** 编译器和 ISA 可以选择不同表示，只需保持可观察语义。
- **Bridge ≠ Integration：** 本册只拥有可复用接口；完整 LOAD 经 Pipeline/Cache/TLB 的过程属于 CO-I01。

9 一页压缩

接口四问

1. 指令从哪些架构状态读取？
 2. 必须产生哪些中间值，它们依赖谁？
 3. 最终只允许写哪些架构状态，PC 怎样变化？
 4. 正常/异常路径分别允许哪些效果提交？
- 回答完四问，CO-02 的语义才能成为 CO-03 可实现、可验证的输入。